CHANGED 3 MONTHS AGO

🗓️ Buổi 12: Object, Giới thiệu jQuery

> Định nghĩa object, truy cập thuộc tính
> Object lồng nhau => Mở rộng nêu khái niệm tham chiếu, tham trị
> Mảng object
> jQuery là gì? Cách nhúng
> So sánh JS thuần vs jQuery
> Cú pháp jQuery
> Fetch

I. Định nghĩa object, truy cập thuộc tính

🧠 1. ĐỊNH NGHĨA OBJECT
✅ Khái niệm:
Trong JavaScript, object là một kiểu dữ liệu dùng để lưu trữ nhiều giá trị liên quan dưới dạng cặp key: value.

```
let user = {
  name: 'An',
  age: 25,
  isStudent: true
};
```

🔹 Từ vựng:

- user: Tên biến object
- name, age, isStudent: thuộc tính (property)
- 'An', 25, true: Giá trị tương ứng

🧠 2. CÁCH TRUY CẬP THUỘC TÍNH
✅ 2.1 Dùng dấu chấm .

```
console.log(user.name);      // 'An'
console.log(user.age);       // 25
```

✅ 2.2 Dùng dấu ngoặc vuông []

```
console.log(user['name']);   // 'An'
```

🔸 Dùng [] khi:

- Tên thuộc tính chứa khoảng trắng
- Tên thuộc tính nằm trong biến

```
let key = 'age';
console.log(user[key]); // 25
```

🧠 3. THÊM – SỬA – XOÁ THUỘC TÍNH
✅ Thêm mới thuộc tính

```
user.email = 'an@example.com';
user['gender'] = 'male';
```

✅ Sửa giá trị

```
user.age = 26;
```

✅ Xoá thuộc tính

```
delete user.isStudent;
```

🧠 4. OBJECT LỒNG NHAU (Nested Object)

```
let person = {
  name: 'Bình',
  address: {
    city: 'Hà Nội',
    street: 'Nguyễn Trãi'
  }
};

console.log(person.address.city); // 'Hà Nội'
```

🧠 5. LẶP QUA CÁC THUỘC TÍNH (Bonus)

```
for (let key in user) {
  console.log(key + ': ' + user[key]);
}
```

🧪 BÀI TẬP THỰC HÀNH
Bài 1:

- Tạo object book với các thuộc tính: title, author, pages.
- In ra tiêu đề sách và tên tác giả

Bài 2:

- Tạo object student gồm: name, scores (object con chứa: math, english).
- In ra điểm tiếng Anh của student.

🎁 GỢI Ý ỨNG DỤNG THỰC TẾ

- Dùng object để lưu thông tin người dùng, sản phẩm, đơn hàng
- Object là nền tảng của JSON → rất quan trọng trong lập trình web

✅ TỔNG KẾT

Kiến thức	Cú pháp	Ghi chú
Khai báo object	`{ key: value }`	Có thể lồng object
Truy cập thuộc tính	`obj.key` hoặc `obj['key']`	Dùng `[]` khi key là biến
Thêm/Sửa/Xoá	`obj.newKey = ...` / `delete obj.key`	

II. Object lồng nhau

🧠 1. OBJECT LỒNG NHAU (NESTED OBJECT)
✅ Khái niệm:
Là object chứa object khác bên trong làm thuộc tính.

```
const user = {
  name: 'An',
  contact: {
    email: 'an@example.com',
    phone: '0123456789'
  }
};
```

✅ Truy cập object lồng nhau:

```
console.log(user.contact.email); // 'an@example.com'
```

🔧 2. TRUY CẬP – THÊM – SỬA – XOÁ THUỘC TÍNH LỒNG
Truy cập:

```
user.contact.phone;         // '0123456789'
```

Thêm thuộc tính:

```
user.contact.address = 'Hà Nội';
```

Sửa:
`user.contact.email = 'new@example.com';`

Xoá:
`delete user.contact.phone;`

⚠️ 3. SỰ KHÁC NHAU GIỮA THAM TRỊ VÀ THAM CHIẾU

Kiểu dữ liệu	Cách lưu trữ	Khi gán biến mới
Nguyên thủy (number, string, boolean, null, undefined)	**Tham trị**	Sao chép giá trị mới
Object, Array, Function	**Tham chiếu**	Trỏ đến cùng địa chỉ vùng nhớ

✅ Ví dụ: Tham trị

```
let a = 5;
let b = a;
b = 10;

console.log(a); // 5 (không bị ảnh hưởng)
```

✅ Ví dụ: Tham chiếu

```
let obj1 = { name: 'Bình' };
let obj2 = obj1;

obj2.name = 'Nam';

console.log(obj1.name); // 'Nam' (bị ảnh hưởng vì cùng tham chiếu)
```

⚠️ Khi sửa object lồng nhau, hãy cẩn thận!

```
const user = {
  name: 'An',
  contact: {
    email: 'an@example.com'
  }
};

const newUser = user;

newUser.contact.email = 'new@example.com';

console.log(user.contact.email); // 'new@example.com' ✅ vì cùng tham chiếu
```

🛡️ 4. SAO CHÉP OBJECT AN TOÀN
❌ Gán trực tiếp (sai):

```
let obj2 = obj1; // => chung vùng nhớ
```

✅ Dùng spread để sao chép nông (shallow copy):

```
let obj2 = { ...obj1 };
```

Nhưng nếu object lồng nhiều cấp → cần deep clone (sao chép sâu)

✅ Cách deep copy đơn giản:

```
let deepCopy = JSON.parse(JSON.stringify(obj1));
```

🎯 5. BÀI TẬP THỰC HÀNH
🔸 Bài 1:
Tạo object student có thuộc tính lồng: info.name, info.age, grades.math, grades.english.

- Truy cập điểm math
- Sửa tên học sinh
- Thêm address vào info
- Sao chép thành một student mới từ student cũ

✅ TỔNG KẾT

Kiến thức	Mô tả
Object lồng nhau	Object chứa object con
Truy cập	Dùng `obj.prop1.prop2` hoặc `obj['prop1']['prop2']`
Tham trị	Gán biến nguyên thủy → tạo bản sao
Tham chiếu	Gán object → 2 biến cùng trỏ 1 vùng nhớ
Giải pháp	Dùng spread `...` hoặc `JSON.parse(JSON.stringify())`

III. Mảng object

🧠 1. MẢNG OBJECT LÀ GÌ?
✅ Khái niệm:

- Là một mảng (array) chứa các object
- Dùng để lưu trữ danh sách các đối tượng có cấu trúc giống nhau

```
const users = [
  { name: 'An', age: 20 },
  { name: 'Bình', age: 25 },
  { name: 'Cường', age: 22 },
];
```

🧠 2. TRUY CẬP VÀ HIỂN THỊ
✅ Truy cập phần tử:

```
console.log(users[0]);          // { name: 'An', age: 20 }
console.log(users[1].name);     // 'Bình'
```

✅ Duyệt mảng:

```
for (let i = 0; i < users.length; i++) {
  console.log(users[i].name);
}

// hoặc dùng forEach:
users.forEach(user => console.log(user.name));
```

🧪 3. BÀI TẬP THỰC HÀNH
🔸 Bài 1:
Tạo mảng products gồm các object:

```
{ name: 'iPhone', price: 1000 }
{ name: 'Laptop', price: 1500 }
{ name: 'Mouse', price: 50 }
```

Yêu cầu:

- In ra tên tất cả sản phẩm
- Thêm sản phẩm mới { name: 'Keyboard', price: 100 }
- Tìm sản phẩm có giá > 1000

🔸 Bài 2:
Tạo mảng students với các object { name, score }.
Yêu cầu:

- Tìm học sinh điểm cao nhất
- Lọc học sinh đỗ (score >= 5)
- Kiểm tra xem có học sinh nào điểm dưới 3 không

VI. JSON

🧠 1. JSON là gì?
✅ Định nghĩa:
JSON (JavaScript Object Notation) là một định dạng dữ liệu dạng văn bản (text), thường dùng để trao đổi dữ liệu giữa client và server.

✅ Đặc điểm:

- Có cấu trúc giống Object trong JS
- Dễ đọc, dễ hiểu
- Hỗ trợ bởi hầu hết ngôn ngữ lập trình

📦 2. Cấu trúc JSON

```
{
  "name": "An",
  "age": 25,
  "isStudent": false,
  "skills": ["HTML", "CSS", "JS"],
  "address": {
    "city": "Hà Nội",
    "zip": "100000"
  }
}
```

🎯 Ghi nhớ:

Thành phần	JSON cho phép
String	✅
Number	✅
Boolean	✅
Array	✅
Object	✅
Null	✅
Function	❌ KHÔNG
Comment (//)	❌ KHÔNG

🔁 3. Chuyển đổi JSON trong JavaScript
✅ JSON.stringify(): Object → JSON (chuỗi)

```
const obj = { name: "An", age: 25 };
const jsonStr = JSON.stringify(obj);
console.log(jsonStr); // '{"name":"An","age":25}'
```

✅ JSON.parse(): JSON (chuỗi) → Object

```
const jsonStr = '{"name":"An","age":25}';
const obj = JSON.parse(jsonStr);
console.log(obj.name); // "An"
```

🌐 4. JSON & Giao tiếp với server (API)
Ví dụ dùng fetch và JSON:

```
fetch('https://api.example.com/user')
  .then(res => res.json()) // chuyển JSON → object
  .then(data => {
    console.log(data.name);
  });

const newUser = {
  name: 'Bình',
  age: 30
};

fetch('https://api.example.com/user', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(newUser) // chuyển object → JSON
});
```

VII. jQuery là gì? Cách nhúng

🧠 1. jQuery là gì?
✅ Khái niệm:

- jQuery là một thư viện JavaScript nhẹ và nhanh, giúp:
- Tương tác với HTML dễ hơn
- Làm hiệu ứng (animations)
- Gửi AJAX dễ hơn
- Xử lý sự kiện đơn giản

✅ Cú pháp cơ bản:

```
$(selector).action();
```

Ví dụ:

```
$('#myButton').click(function () {
  alert('Đã click!');
});
```

📌 2. Vì sao jQuery từng rất phổ biến?

Tính năng	Lợi ích
Viết ngắn gọn	Dễ dùng, cú pháp ngắn hơn so với JavaScript thuần
Hỗ trợ trình duyệt	Hoạt động ổn định trên nhiều trình duyệt cũ
AJAX đơn giản	Gửi/nhận dữ liệu không cần tải lại trang rất dễ dàng
DOM thao tác tốt	Dễ chọn và thay đổi phần tử HTML

🧠 3. Cách nhúng jQuery vào trang web
✅ Cách 1: Dùng CDN (nhanh gọn)
Chèn đoạn này vào phần <head> hoặc cuối <body>:

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

✅ Cách 2: Tải file jQuery về
Truy cập: https://jquery.com/download
Tải file .js về
Nhúng vào HTML:

```
<script src="js/jquery-3.6.0.min.js"></script>
```

🧪 4. Ví dụ đơn giản

```
<!DOCTYPE html>
<html>
<head>
  <title>Demo jQuery</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
  <button id="myBtn">Click me</button>
  <p id="text">Hello!</p>

  <script>
    $('#myBtn').click(function () {
      $('#text').text('Bạn đã click nút!');
    });
  </script>
</body>
</html>
```

⚠️ 5. Lưu ý khi dùng jQuery

- jQuery không còn quá cần thiết trong các project mới vì:
- JavaScript hiện đại (ES6+) đã dễ dùng hơn
- Các thư viện như React, Vue, Angular... thay thế jQuery
- Tuy nhiên, jQuery vẫn còn trong nhiều hệ thống cũ, template, WordPress,...

✅ TỔNG KẾT

Nội dung	Ghi nhớ
jQuery là gì	Thư viện JS giúp viết code đơn giản hơn
Cú pháp cơ bản	`$(selector).action()`
Nhúng jQuery	Qua CDN hoặc tải file về
Khi nào dùng	Dự án cũ, nhỏ, học cơ bản

🎯 Gợi ý bài tập thực hành
Bài 1:

- Tạo 1 nút "Ẩn/Hiện"
- Click để ẩn hoặc hiện một đoạn văn

VIII. So sánh JS thuần vs jQuery

🧠 1. Giới thiệu ngắn

Thuật ngữ	Giải thích ngắn
JS thuần	Là JavaScript gốc, không dùng thư viện nào
jQuery	Là thư viện viết bằng JavaScript để rút gọn và đơn giản hóa thao tác

🧪 2. So sánh qua ví dụ thực tế
✅ 2.1 Chọn phần tử HTML

Công việc	jQuery	JS thuần
Chọn theo ID	`$('#title')`	`document.getElementById('title')`
Chọn theo class	`$('.item')`	`document.querySelectorAll('.item')`
Chọn theo tag	`$('p')`	`document.getElementsByTagName('p')`

✅ 2.3 Thay đổi nội dung

Công việc	jQuery	JS thuần
Đổi nội dung văn bản	`$('#text').text('Hi')`	`document.getElementById('text').textConte` `= 'Hi'`
Đổi HTML	`$('#text').html('<b>Hi</b>')`	`document.getElementById('text').innerHTML` `'<b>Hi</b>'`

✅ 2.4 Thay đổi CSS

Công việc	jQuery	JS thuần
Đổi nội dung văn bản	`$('#text').text('Hi')`	`document.getElementById('text').textConte` `= 'Hi'`
Đổi HTML	`$('#text').html('<b>Hi</b>')`	`document.getElementById('text').innerHTML` `'<b>Hi</b>'`

✅ 2.5 Thêm/Xoá class

Công việc	jQuery	JS thuần
Thêm class	`$('#box').addClass('active')`	`element.classList.add('active')`
Xoá class	`$('#box').removeClass('active')`	`element.classList.remove('active')`
Toggle class	`$('#box').toggleClass('active')`	`element.classList.toggle('active')`

📊 3. Ưu điểm & nhược điểm
✅ jQuery – Ưu điểm

- Cú pháp ngắn gọn, dễ viết
- Dễ học với người mới
- Hỗ trợ trình duyệt cũ
- Có sẵn nhiều hiệu ứng (fadeIn, slideUp,...)

❌ jQuery – Nhược điểm

- Cần tải thêm thư viện (nặng hơn)
- Không phù hợp với project lớn/modern
- Ít dùng trong framework hiện đại (React, Vue, Angular)

✅ JavaScript thuần – Ưu điểm

- Không cần cài thêm gì
- Tối ưu hơn, chạy nhanh hơn
- Dễ tích hợp với framework hiện đại

❌ JS thuần – Nhược điểm

- Cú pháp dài hơn với người mới
- Trước đây thiếu một số tính năng (hiện tại ES6 đã cải thiện rất nhiều)

IX. Cú pháp JQuery

🧠 1. jQuery hoạt động như thế nào?
✅ Cú pháp chung:

```
$(selector).action();
```

Thành phần	Giải thích
`$`	Gọi hàm jQuery
`selector`	Bộ chọn phần tử HTML (như CSS)
`action()`	Hành động thực hiện (click, hide, css, ...)

🧠 2. CÁCH CHỌN PHẦN TỬ

```
$('#title') // Theo Id
$('.item') // Theo class
$('p')  // Theo tag
```

🧠 3. THAO TÁC SỰ KIỆN
✅ 3.1. Sự kiện chuột (Mouse Events)

Sự kiện jQuery	Ý nghĩa	Ví dụ
`.click()`	Khi click vào phần tử	`$('#btn').click(function () {})`
`.dblclick()`	Khi double-click	`$('#box').dblclick(function () {})`
`.mouseenter()`	Khi chuột đi vào phần tử	`$('#box').mouseenter(function () {})`
`.mouseleave()`	Khi chuột rời khỏi phần tử	`$('#box').mouseleave(function () {})`
`.mousedown()`	Khi nhấn nút chuột (chưa nhả)	`$('#box').mousedown(function () {})`
`.mouseup()`	Khi nhả nút chuột	`$('#box').mouseup(function () {})`
`.mousemove()`	Khi chuột di chuyển bên trong phần tử	`$('#box').mousemove(function () {})`
`.hover(in, out)`	Khi chuột vào và ra (gộp 2 sự kiện)	`$('#box').hover(inFunc, outFunc)`

✅ 3.2. Sự kiện bàn phím (Keyboard Events)

Sự kiện jQuery	Ý nghĩa	Ví dụ
`.keydown()`	Khi nhấn một phím (bất kỳ)	`$('#input').keydown(function () {})`
`.keyup()`	Khi nhả một phím	`$('#input').keyup(function () {})`
`.keypress()`	Khi nhấn phím in ra ký tự (trừ phím đặc biệt)	`$('#input').keypress(function () {})`

✅ 3.3. Sự kiện biểu mẫu (Form Events)

Sự kiện jQuery	Ý nghĩa	Ví dụ
`.submit()`	Khi form được gửi	`$('#form').submit(function (e) { e.preventDefault(); })`
`.change()`	Khi giá trị input/select thay đổi	`$('#select').change(function () {})`
`.focus()`	Khi phần tử được focus vào	`$('#input').focus(function () {})`
`.blur()`	Khi focus rời khỏi phần tử	`$('#input').blur(function () {})`
`.select()`	Khi người dùng chọn text trong input	`$('#input').select(function () {})`
`.input()`	Khi nội dung input thay đổi (dùng `.on('input', fn)`)	`$('#input').on('input', function () {})`

✅ 3.4. Sự kiện tài liệu & trình duyệt

Sự kiện jQuery	Ý nghĩa	Ví dụ
`.ready()`	Khi DOM đã tải xong	`$(document).ready(function () {})`
`.resize()`	Khi cửa sổ trình duyệt thay đổi kích thước	`$(window).resize(function () {})`
`.scroll()`	Khi cuộn trang	`$(window).scroll(function () {})`
`.load()` ❌ (đã bỏ)	Trước đây dùng cho ảnh/tài liệu tải xong	

✅ 3.5. Khai báo bằng .on() (cách hiện đại, linh hoạt)
Bạn có thể dùng .on() để gán bất kỳ sự kiện nào:

```
$('#btn').on('click', function () { ... });
$('#form').on('submit', function () { ... });
$('#input').on('keyup', function () { ... });
```

🧪 Ví dụ gộp tất cả:

```
$(document).ready(function () {
  $('#btn').click(function () {
    alert('Bạn đã click');
  });

  $('#form').submit(function (e) {
    e.preventDefault();
    alert('Đã submit');
  });

  $('#input').keyup(function () {
    console.log('Đang gõ...');
  });

  $('#box').hover(
    function () {
      $(this).css('background', 'lightblue');
    },
    function () {
      $(this).css('background', '');
    }
  );
});
```

🧠 4. THAY ĐỔI NỘI DUNG

```
$('#text').text('Xin chào!'); // Thay đổi văn bản

$('#text').html('<b>Xin chào!</b>'); // Thay đổi HTML

let name = $('#nameInput').val(); // Lấy giá trị input
```

🧠 5. THAY ĐỔI CSS

```
$('#box').css('background-color', 'red');
$('#box').css({
  color: 'white',
  fontSize: '20px'
});
```

🧠 6. THÊM/XÓA CLASS

```
$('#box').addClass('active');
$('#box').removeClass('active');
$('#box').toggleClass('active');
```

🧠 7. ẨN/HIỆN – HIỆU ỨNG

```
$('#box').hide();
$('#box').show();
$('#box').fadeIn();
$('#box').fadeOut();
$('#box').slideUp();
$('#box').slideDown();
```

✅ TỔNG KẾT

Tác vụ	Cú pháp jQuery
Chọn phần tử	`$('#id'), $('.class'), $('tag')`
Gán sự kiện	`.click(function() {})`
Thay nội dung	`.text()` , `.html()` , `.val()`
Thay CSS	`.css('property', 'value')`
Thêm/xoá class	`.addClass()` , `.removeClass()`
Ẩn/Hiện phần tử	`.hide()` , `.show()` , `.fadeIn()`
Vòng lặp qua selector	`.each(function() {})`

🧠 GỢI Ý MỞ RỘNG
Dùng $(document).ready() để đảm bảo DOM đã tải:

```
$(document).ready(function () {
  // Code jQuery ở đây
});
```

X. Fetch

🧠 1. Fetch là gì?
✅ Định nghĩa:
fetch() là một hàm dựng sẵn trong JavaScript, dùng để gửi yêu cầu HTTP tới server (API) và nhận dữ liệu trả về.

- Thay thế dần XMLHttpRequest
- Dễ dùng hơn, cú pháp hiện đại hơn (dùng Promise)

🧠 2. Cú pháp cơ bản

```
fetch(url, options)
  .then(response => response.json())
  .then(data => {
    // xử lý data ở đây
  })
  .catch(error => {
    console.error('Có lỗi:', error);
  });
```

```
* url: đường dẫn API
* options: tuỳ chọn như method, headers, body,...
* .then(): xử lý khi thành công
* .catch(): xử lý lỗi nếu có
```

🧪 Ví dụ 1: Gửi yêu cầu GET

```
fetch('https://jsonplaceholder.typicode.com/posts')
  .then(res => res.json())
  .then(data => {
    console.log(data); // mảng bài viết
  })
  .catch(err => console.error(err));
```

🧪 Ví dụ 2: Gửi yêu cầu POST

```
fetch('https://jsonplaceholder.typicode.com/posts', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    title: 'Tiêu đề mới',
    body: 'Nội dung bài viết',
    userId: 1
  })
})
.then(res => res.json())
.then(data => {
  console.log('Tạo thành công:', data);
})
.catch(err => console.error(err));
```

🧠 3. Các phương thức HTTP thường dùng

Phương thức	Mục đích
`GET`	Lấy dữ liệu
`POST`	Gửi dữ liệu mới
`PUT/PATCH`	Cập nhật dữ liệu
`DELETE`	Xoá dữ liệu

⚠️ 4. Xử lý lỗi khi fetch

```
fetch(url)
  .then(res => {
    if (!res.ok) throw new Error('Lỗi HTTP: ' + res.status);
    return res.json();
  })
  .then(data => console.log(data))
  .catch(err => console.error('Lỗi xảy ra:', err.message));
```

💡 5. Fetch với async/await (cách hiện đại)

```
async function loadData() {
  try {
    const res = await fetch('https://jsonplaceholder.typicode.com/posts');
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.error('Lỗi:', err);
  }
}

loadData();
```

🧪 Bài tập thực hành
Bài 1: Gọi API GET

- Dùng fetch() để lấy danh sách người dùng từ https://jsonplaceholder.typicode.com/users
- In danh sách người dùng ra màn hình dưới dạng table
- Mỗi hàng sẽ có một button xem chi tiết, khi click vào button sẽ gọi API lấy chi tiết user